

Diseño e Integración de un Sistema de Cómputo Basado en el Microprocesador Z80

[Flores Hinojosa Ricardo]
Universidad Nacional Autónoma de México
Facultad de Estudios Superiores Cuautitlán

24 de septiembre de 2026

Resumen

El presente proyecto detalla la integración y diseño de un sistema basado en el microprocesador Z80. El diseño abarca la definición del mapa de memoria (EEPROM y SRAM) y puertos de comunicación paralela, finalizando con la implementación de rutinas a nivel ensamblador para la interacción con el usuario mediante la lectura, validación y conteo de caracteres alfanuméricos. El objetivo es demostrar la correcta abstracción entre los elementos de hardware y las rutinas de software a bajo nivel.

1 Introducción

A pesar del tiempo transcurrido desde su creación, el microprocesador Z80 se mantiene como una herramienta académica e industrial clave para comprender el diseño de arquitecturas computacionales a nivel de registros y buses. En el entorno universitario, su estudio permite aterrizar conceptos abstractos de la electrónica digital.

Este escrito fundamenta el funcionamiento del hardware propuesto, el papel de las memorias de almacenamiento y los periféricos de interfaz, consolidando el trabajo desarrollado durante la última semana en un esquema electrónico y analizando los resultados del código ejecutable.

2 Funcionamiento del sistema

2.1 Microprocesador Z80

El Z80 es una CPU de 8 bits equipada con un bus de direcciones de 16 bits, lo que le permite controlar de forma nativa hasta 64 KB de memoria y 256 puertos de Entrada/Salida (E/S). El funcionamiento en este diseño recae en la sincronización de las señales del bus de control, principalmente $\overline{MREQ}$ para direccionar dispositivos de memoria, $\overline{IORQ}$ para la activación de periféricos, y las líneas $\overline{RD}$ y $\overline{WR}$ para indicar operaciones de lectura o escritura de datos.

2.2 Memorias del Sistema

El sistema consta de dos bloques de memoria de 16 KB cada uno, divididos de acuerdo con los requerimientos de almacenamiento:

- **Memoria no volátil (EEPROM):** Inicia en la dirección $0000h$ y se utiliza para almacenar permanentemente las instrucciones y textos (el código ensamblador ya ensamblado y cargado).
- **Memoria volátil (SRAM):** Mapeada lógicamente a partir de la dirección $C000h$ mediante un circuito de decodificación. Almacena los datos transitorios, tales como el buffer de caracteres de entrada del usuario y el seguimiento de la pila (*Stack*) del sistema.

2.3 Interfaz para periféricos

Para lograr una comunicación efectiva en paralelo, se configuran puertos mapeados en memoria o en el espacio de E/S (a partir de la dirección $40h$). En este tipo de diseños compatibles, se suele utilizar el controlador Z80 PIO (*Parallel Input/Output*). Este componente permite configurar puertos bidireccionales con control de *handshake* gestionado mediante interrupciones, facilitando enormemente la interfaz física para dispositivos externos como un teclado matricial o el controlador de una pantalla LCD alfanumérica.

3 Diseño del sistema

El diagrama electrónico estructurado para el proyecto plantea la conexión del bus de datos ($D_0 - D_7$) de forma bidireccional, interconectando la CPU, las memorias y los controladores de puerto.

Por otro lado, el bus de direcciones ($A_0 - A_{15}$) ataca directamente a las memorias para localizar bytes específicos y retroalimenta la lógica de decodificación. Esta lógica, implementada mediante el circuito integrado decodificador **74LS138**, es la encargada de generar las señales de habilitación (Chip Select, $\overline{CS}$). El uso de este decodificador asegura que nunca existan conflictos de colisión en el bus de datos cuando la CPU cambia del espacio de ejecución

de la EEPROM a la SRAM, o cuando realiza llamadas hacia los registros de los periféricos.

manera controlada y sin fallos de desbordamiento en la memoria volátil.

4 Análisis de los resultados

La lógica generada en el código ensamblador demuestra que el hardware es capaz de interpretar los estados de los puertos periféricos e implementar una validación robusta de manera eficiente.

Al cargar las direcciones base, la CPU lee bytes individuales. Para procesar esta información, se utilizaron extensamente las instrucciones lógicas de comparación (CP) y desvío condicional (JR C, JR Z) para filtrar la información en tiempo real.

A continuación se muestra un fragmento representativo de la rutina implementada para la lectura y validación:

```
; Rutina de validacion alfanumerica
READ_PORT:
    IN A, (40h)    ; Leer dato del periférico
    CP 20h        ; Comparar con 'Espacio'
    JR Z, IS_SPC  ; Saltar si es espacio

    CP 41h        ; Comparar con 'A'
    JR C, INVALID ; Rechazar si es menor

    CP 5Bh        ; Comparar con 'Z' + 1
    JR NC, INVALID; Rechazar si es mayor o igual

VALID_CHAR:
    INC B         ; Incrementar contador grafemas
    ; ... guardar en SRAM ...
```

Figura 1: Fragmento de rutina en ensamblador Z80.

Como se observa en el código, el sistema bloquea eficazmente cualquier carácter distinto al alfabeto (como símbolos o números no deseados) y registra de manera aislada los espacios. Esto asegura un conteo preciso y estricto únicamente de los grafemas del nombre introducido por el usuario, evitando corrupciones en el *buffer* de datos.

5 Conclusión

El desarrollo integral de este sistema Z80 durante el proyecto evidencia que la abstracción entre hardware y software es muy estrecha y dependiente. Diseñar correctamente la circuitería lógica para el mapeo de memoria y periféricos es tan crítico como estructurar limpiamente el código ensamblador.

El correcto direccionamiento de los buses implementado con el 74LS138 ha garantizado que el programa pueda interactuar con el entorno físico de

Referencias

- [1] ZiLOG, Inc. (2001). *Z80 Family CPU Peripherals User Manual* (UM008101-0601). San Jose, CA.
- [2] Gaonkar, R. S. (1993). *Microprocessor Architecture, Programming, and Applications with the 8085*. Penram International.